

UT1.1 · Bash

Funciones — Ejercicios de repaso

Practicar funciones repasando parámetros, matemáticas, condicionales, bucles, entrada de datos y utilidades

Índice

Cinco actividades independientes, de menor a mayor dificultad.

- 1. bytes_a_gb.sh** — *Funciones + Matemáticas (bc)*
- 2. clasificar_http.sh** — *Funciones + Condicionales*
- 3. tipos_fichero.sh** — *Funciones + Bucles + Utilidades (find)*
- 4. pedir_puerto_valido.sh** — *Funciones + Entrada de datos + Bucles*
- 5. existe_usuario.sh** — *Funciones + Gestión de usuarios*

Instrucciones

- Trabajaréis sobre la misma carpeta ~/prueba_bash de siempre.
- Cada actividad es independiente: podéis saltar a la siguiente aunque no hayáis terminado la anterior.
- Antes de dar una actividad por cerrada, comprobadla con los casos de su "Lista de verificación".
- Cuando deis una actividad por cerrada, subid el script a vuestro repositorio individual de ASO, dentro de la carpeta bash/sesion2_funciones/.

Actividad 1. bytes_a_gb.sh

Repasa: Funciones + Matemáticas (bc)

Escribe una función bytes_a_gb que reciba un número de bytes (por ejemplo, el tamaño que devuelve du o df) y devuelva (con echo) su equivalente en GB, con 2 decimales de precisión. 1 GB = 1073741824 bytes.

Pista

```
echo "scale=2; $bytes / 1073741824" | bc — scale=2 fija dos decimales de precisión.
```

Lista de verificación:

- bytes_a_gb 5000000000 → 4.65
- bytes_a_gb 1073741824 → 1.00

Actividad 2. clasificar_http.sh

Repasa: Funciones + Condicionales

Escribe una función clasificar_http que reciba un código de estado HTTP (como los que aparecen en los logs de un servidor web) y devuelva (con echo): "Éxito" (200-299), "Redirección" (300-399), "Error del cliente" (400-499), "Error del servidor" (500-599). Prueba la función con varios códigos.

Lista de verificación:

- clasificar_http 200 → Éxito
- clasificar_http 301 → Redirección
- clasificar_http 404 → Error del cliente
- clasificar_http 500 → Error del servidor
- Usa elif para encadenar los cuatro tramos, con [[]].

Actividad 3. tipos_fichero.sh

Repasa: Funciones + Bucles + Utilidades (find)

Escribe una función contar_por_extension que reciba una carpeta y una extensión (por ejemplo, log o txt), y devuelva (con echo) cuántos ficheros de esa extensión contiene. Con un bucle, aplica la función a ~/prueba_bash/datos para las extensiones log, txt y csv, y muestra el resultado de cada una.

Pista

```
find "$carpeta" -maxdepth 1 -type f -name "*.Sext" | wc -l cuenta los ficheros que coinciden.
```

Lista de verificación:

- log → 2
- txt → 2
- csv → 1

Actividad 4. pedir_puerto_valido.sh

Repasa: Funciones + Entrada de datos + Bucles

Escribe una función pedir_puerto_valido que pida repetidamente un puerto TCP (con read -p) hasta que el usuario introduzca un número entre 1 y 65535, y entonces termine. Llama a la función y, cuando acabe, muestra el puerto recibido.

Pista

`[[ $puerto =~ ^[0-9]+$ ]]` comprueba que la cadena son solo dígitos; `(( puerto >= 1 && puerto <= 65535 ))` comprueba que está en rango.

Si no declaras puerto con local dentro de la función, sigue disponible fuera de ella al terminar — no hace falta devolverlo con echo.

Lista de verificación:

- Al introducir "abc" o "99999", el script avisa y vuelve a pedir el puerto.
- Al introducir "8080", el script termina y muestra: Puerto válido recibido: 8080

Actividad 5. existe_usuario.sh

Repasa: Funciones + Gestión de usuarios

Escribe una función existe_usuario que reciba un nombre de usuario y devuelva (con echo) si existe o no en el sistema. Llama a la función con un usuario que exista (por ejemplo, root) y con uno que no.

Pista

`if id "$1" &>/dev/null; then ...` comprueba si el usuario existe: id falla si no lo encuentra, y `&>/dev/null` descarta su salida.

Lista de verificación:

- `existe_usuario root` → El usuario root existe en el sistema.
- `existe_usuario noexiste123` → El usuario noexiste123 no existe.

Cuando termines una actividad, avisa al profesor/a para revisarla antes de pasar a la siguiente.